

一、整体业务流程

前端提供聊天输入和文件上传，后端根据用户请求进入三个分支：

分支	用户需求示例	实现方式
领域知识问答	“PWL-RTDETR 中的小波卷积有什么作用？”	问答 Agent 查询知识库，组织有来源的回答
目标检测	上传红外图片，点击“开始检测”	Python 节点调用 PWL-RTDETR，绘制预测结果并返回
知识库管理	“把这份新版模型说明加入知识库，检查是否有旧版本。”	知识库管理 Agent 查询、比较并调用入库或更新工具

明确的“开始检测”按钮可以直接进入检测节点；聊天中的自然语言请求，由意图分类节点结合用户输入和必要的历史上下文进行路由。

二、两个 Agent 的职责

****问答 Agent 负责“使用知识”。****通过 `create_agent` 配置模型、检索工具和 `Middleware`。它根据问题生成检索 `query`，获取文档片段；资料不足时，可以改写查询继续检索，最后输出回答与引用来源，并设置调用次数上限。

如果用户追问某次检测结果，可以额外提供检测结果查询工具，让 **Agent** 读取任务 `JSON`。纯文本 `LLM` 只能理解这些结构化结果，不能直接看懂预测图片，也不能据此断定漏检原因。

****知识库管理 Agent 负责“维护知识”。****接收用户上传的文档和管理指令，自主选择工具完成：

1. 查找相似文档或旧版本。
2. 读取候选内容，判断是否重复、是否存在版本差异。
3. 决定新增、提出更新建议，或向用户澄清。
4. 用户确认覆盖后，执行更新。
5. 检索新内容，检查入库结果是否可用。

第一版限定 **Markdown / TXT**，支持查重、新增和确认后更新，控制开发范围。权限校验由后端代码执行，不能只靠提示词约束。

这两个 **Agent** 可以使用同一个底层 `LLM`，但配置不同的提示词、工具集合和上下文。

三、目标检测节点

FastAPI 保存上传图片，将图片引用写入 State。检测节点读取图片，调用已加载的 **PWL-RTDETR**，得到检测框、类别和置信度，再通过 OpenCV 或 Pillow 绘制预测图。

后端保存结果图，返回**图片访问 URL、任务 ID 和检测 JSON**，前端加载图片展示。推理和绘图均由 Python 执行，不需要 LLM。

四、RAG 与公共工程能力

模块	具体设计
知识入库	文档解析、清洗、内容哈希去重、分块、Embedding，写入 PostgreSQL + pgvector，并保留来源信息
知识检索	向量检索；按原先设想加入 BM25 混合召回与 RRF 融合排序，通过评测验证收益
Middleware	按任务与用户上下文动态组装提示词，通过 wrap_tool_call 记录工具参数、耗时、结果和异常
短期记忆	使用 PostgresSaver 保存会话状态；新会话生成雪花 thread_id，后续请求复用
长期记忆	使用 PostgresStore 保存用户明确的偏好等信息，按用户隔离并按需读取
失败处理	临时错误有限重试；输入错误反馈用户；结合持久化检查点实现恢复入口
写入可靠性	文档更新记录版本，重复请求避免重复入库，失败恢复避免重复覆盖

服务启动时初始化 Agent、检测模型和数据库资源，并编译图；每次请求复用这些对象。内部 Agent 执行到哪一步可以恢复，需要结合子图与 Checkpointer 配置实际验证，不能仅凭外层配置就宣称所有工具步骤都不会重跑。

无人机红外小目标检测与知识管理智能体

****项目描述：****面向无人机红外小目标检测场景，基于自研 PWL-RTDETR 与领域知识库构建智能助手，支持图像检测、知识问答及自然语言驱动的知识库维护，通过 LangGraph 编排专业 Agent 与检测流程。

****技术栈：******Python、FastAPI、LangGraph、LangChain、PostgreSQL、pgvector、PWL-RTDETR、Docker compose**

主要工作：

- 1. ****流程编排与模型接入：********基于 LangGraph 构建意图识别与条件路由，编排问答 Agent、知识库管理 Agent 和检测节点；通过 FastAPI 接收红外图片，调用 PWL-RTDETR 完成推理，输出结构化检测结果及可视化预测图。
- 2. ****领域知识问答：********构建文档解析、去重、分块、向量化与入库流程，结合向量检索、BM25 与 RRF 实现混合检索；由问答 Agent 按需改写查询、补充检索，生成附带资料来源的领域回答。
- 3. ****知识库管理 Agent：********将文档搜索、正文读取、新增、更新及检索验证封装为工具，支持根据自然语言指令查找旧版本、比较内容并执行知识维护；结合用户确认、后端权限校验与文档版本控制，避免未经确认的覆盖及重复入库。
- 4. ****持久化与调用治理：********基于 PostgresSaver 与会话标识实现多轮状态持久化及失败恢复，利用 PostgresStore 保存跨会话偏好；通过 LangChain Middleware 实现动态提示词与工具调用追踪，配置超时、有限重试和调用次数限制。

概念图（只是提供思路）：

